MACHINE LEARNING

UINIT-I

Learning

- Learning refers to the ability to **improve performance through experience**.
- In machine learning, learning mainly means **learning from data**, since data is widely available in large quantities.
- Human learning can be understood as **learning from experience**, where past situations guide future actions.
- Learning provides **flexibility and adaptability**, allowing humans (and machines) to adjust to new situations.
- Important aspects of learning are:
    - **Remembering** past experiences
    - **Adapting** actions based on success or failure
    - **Generalising**, i.e., applying knowledge from one situation to similar new situations
- Intelligence also includes reasoning and logical deduction, but machine learning mainly focuses on:
    - Learning
    - Adapting behaviour
- Early Artificial Intelligence focused on **symbolic processing**, where computers manipulated symbols representing the environment.
- Machine learning methods are often called **sub-symbolic**, as they do not explicitly use symbols or symbolic reasoning.

Machine Learning and Types of Machine Learning

- Machine learning is the process of enabling computers to **modify or adapt their actions** to improve accuracy.

- These actions may include:
    - Making predictions
    - Controlling machines or robots
- Accuracy is measured by how closely the computer's output matches the correct output.
- Example:
    - A computer playing a game like Scrabble may initially lose.
    - After playing many games, it improves and starts winning.
    - This improvement indicates that the computer is learning.
- Once trained, the computer can apply the same strategies to new players, which shows **generalisation**.
- Machine learning is a **multi-disciplinary field**, combining ideas from:
    - Biology and neuroscience
    - Statistics and mathematics
    - Physics and computer science
- The human brain is proof that learning systems are possible, inspiring many ML techniques.
- Neural networks were inspired by biological neurons and later reinterpreted as statistical models.
- Another major influence on machine learning is **data mining**, which focuses on extracting useful information from large datasets.
- **Computational complexity** is an important concern in machine learning:
    - Training complexity: time taken to train the model
    - Testing complexity: time taken to make predictions
- Training can take longer since it is done less frequently.
- Prediction (testing) must be fast, as it is done many times in real-world applications.

Types of Machine Learning

1. Supervised Learning

- The algorithm is trained using a **training dataset with correct outputs (targets)**.
- The goal is to learn a mapping from inputs to outputs.
- The model generalises to give correct outputs for unseen inputs.
- Also known as **learning from examples**.
- Most commonly used type of machine learning.

2. Unsupervised Learning

- No correct outputs are provided.
- The algorithm finds **patterns or similarities** among the input data.
- Inputs with similar characteristics are grouped together.
- Often used for clustering and pattern discovery.
- Statistical unsupervised learning is known as **density estimation**.

3. Reinforcement Learning

- The algorithm is told whether its action is **right or wrong**, but not how to correct it.
- It learns by **trial and error**.
- A score or reward may be given instead of a simple right/wrong signal.
- Sometimes called **learning with a critic**.

4. Evolutionary Learning

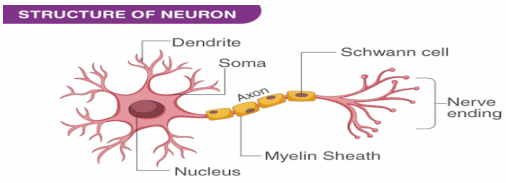
- Inspired by **biological evolution**.
- Solutions evolve over time to improve performance.
- Uses the concept of **fitness**, which measures how good a solution is.
- Better solutions survive and produce improved versions.

The Brain and the Neuron

The Brain

- In animals, **learning takes place in the brain**.

- Understanding how the brain works can help us **design better machine learning systems**.
- Although the brain is highly complex, its **basic building blocks are simple**.
- The brain:
    - Handles **noisy and inconsistent data**
    - Processes **high-dimensional data** (e.g., images)
    - Produces answers **very quickly**
- The human brain weighs about **1.5 kg** and loses neurons over time.
- This property is called **robustness**.
- We focus only on the **basic processing units of the brain**, not higher-level cognition.
- The basic processing units of the brain are called **neurons**.
- The brain contains approximately **100 billion (10^{11}) neurons**.
- Neurons may differ in type, but their **basic functioning is similar**.



Working of a Biological Neuron

- Chemical transmitters affect the **electrical potential** inside the neuron.
- If the membrane potential reaches a **threshold**, the neuron:
    - Fires (spikes)
    - Sends a pulse of fixed strength and duration
- The signal travels through an **axon**.
- Axons branch out and connect to other neurons through **synapses**.
- Each neuron is connected to **thousands of other neurons**.
- The brain contains approximately **100 trillion (10^{1}) synapses**.
- After firing, a neuron needs a **refractory period** before firing again.
- Each neuron performs a simple computation:
    - Decides whether to fire or not

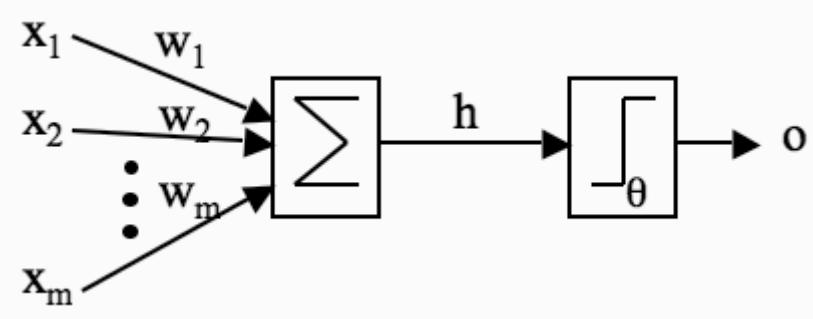
- The brain is a **massively parallel system** with billions of processing units.
- This idea leads to the concept of **Strong AI**, which aims to replicate human intelligence.
- This book focuses only on **learning systems**, not full human intelligence.

Artificial Neuron: McCulloch and Pitts

- Studying real neurons is difficult because:
  - Neurons are very small
  - Extracting and keeping them alive is challenging
- Hodgkin and Huxley (1952):
  - Studied neurons from giant squid
  - Developed detailed mathematical equations
  - Won a Nobel Prize
- Instead of biological detail, we use a **simplified mathematical model**.
- McCulloch and Pitts proposed a neuron model in **1943**.
- The goal of the model:
  - Capture essential behavior
  - Ignore biological complexity

Components of McCulloch–Pitts Neuron

- A McCulloch–Pitts neuron consists of:
  1. **Weighted inputs** (represent synapses)
  2. **Adder** (sums all input signals)
  3. **Activation function** (decides whether the neuron fires)



- Inputs: x1,x2,...,xm
- Weights: w1,w2,...,wm
- Weighted sum:

$$h = \sum_{i=1}^{m} w_i x_i$$

- Threshold value: θ
- Activation function:
    - Output = 1 if h > θ
    - Output = 0 if h ≤ θ
- The neuron is a **binary threshold device**.
- It either:
    - Fires (output = 1)
    - Does not fire (output = 0)

Importance of McCulloch–Pitts Neurons

- Though simple, these neurons are powerful.
- Networks of such neurons can:
    - Perform any computation a computer can
    - Learn functions
    - Classify data
    - Memorize patterns
- Learning mainly involves **adjusting the weights**.
- Most neural network learning algorithms focus on weight selection.

Limitations of McCulloch and Pitts Model

- The model is **not biologically realistic**.
- Limitations include:
    - Real neurons do not sum inputs linearly
    - Real neurons produce **spike trains**, not single outputs

    - Outputs are continuous, not binary
    - Firing thresholds change over time
    - Neurons update asynchronously (not clock-based)
- Real neurons:
    - Use chemical processes
    - Have varying neurotransmitter levels
- In models:
    - Neurons are often updated synchronously
- Weights in models:
    - Can be positive or negative
- Biological neurons:
    - Are either excitatory or inhibitory
    - Do not switch between the two
- Real neurons can have **feedback connections** to themselves.
- Such feedback is usually not allowed in basic models.

Designing a Learning System

- A learning system is a program that **improves its performance through experience**.
- Designing a learning system requires answering key questions:
    - What task should the system learn?
    - What experience will it learn fconrom?
    - How will performance be measured?
- Main components of a learning system:
    - **Performance element** – makes decisions or predictions
    - **Learning element** – improves the performance element
    - **Critic** – evaluates performance
    - **Problem generator** – suggests new experiences
- Example:
    - Email spam filter

- Performance: accuracy of classification

Perspectives and Issues in Machine Learning

Different Perspectives

- **Statistical perspective** – learning as function approximation from data
- **Computational perspective** – focuses on efficiency and algorithm complexity
- **Biological perspective** – inspired by brain and neurons
- **Psychological perspective** – models human learning behavior

Important Issues

- Choosing the right **training data**
- Selecting appropriate **features**
- Avoiding **overfitting and underfitting**
- Handling **noise** in data
- Computational complexity
- Generalization to unseen data
- Ethical issues such as bias and privacy

Concept Learning Task – Concept Learning as Search

Concept Learning Task

- Concept learning is the task of **inferring a boolean-valued function** from training examples.
- Each example is described by a set of **attributes**.
- The target concept classifies examples as:
  - Positive (Yes)

    - Negative (No)
- Goal:
    - Learn a hypothesis that correctly classifies all training examples
- Example:
    - Learning the concept of "play tennis" based on weather conditions
- Concepts are represented as:
    - Conjunctions of attribute values

Concept Learning as Search

- Concept learning can be viewed as a **search problem**.
- Search space consists of all possible hypotheses.
- The learner searches for a hypothesis that:
    - Fits all positive examples
    - Excludes all negative examples
- Search starts from:
    - Most general hypothesis
    - Or most specific hypothesis
- Learning is equivalent to **searching the hypothesis space**.

Finding a Maximally Specific Hypothesis

- A **maximally specific hypothesis**:
    - Covers all positive examples
    - Excludes as many negative examples as possible
- The **Find-S algorithm** is used.
- Steps of Find-S algorithm:

    1. Initialize hypothesis to the most specific hypothesis

2. For each positive example:
    - Generalize the hypothesis minimally to include the example
3. Ignore negative examples

- Advantages:
    - Simple and efficient
- Limitations:
    - Cannot handle noise
    - Ignores negative examples

Version Spaces

- Version space is the set of **all hypotheses consistent with training data**.
- It represents all possible correct hypotheses.
- Version space is bounded by:
    - **S boundary** – set of maximally specific hypotheses
    - **G boundary** – set of maximally general hypotheses
- Any hypothesis between S and G is consistent with the data.
- As more examples are seen:
    - Version space shrinks
    - Uncertainty reduces

Candidate Elimination Algorithm

- Candidate Elimination algorithm computes the **version space**.
- It maintains two sets:
    - **S** – maximally specific hypotheses
    - **G** – maximally general hypotheses

Algorithm Steps

- Initialize:
    - S = most specific hypothesis
    - G = most general hypothesis
- For each training example:
    - If example is **positive**:
        - Remove from G hypotheses that do not cover it
        - Generalize S minimally
    - If example is **negative**:
        - Remove from S hypotheses that cover it
        - Specialize G minimally
- Continue until all examples are processed

Advantages

- Uses both positive and negative examples
- Maintains all consistent hypotheses

Limitations

- Sensitive to noise
- Computationally expensive
- Assumes hypothesis space is correct

Linear Discriminants: Linear Separability

Linear Discriminants

- Linear discriminants are classification methods that **separate data using a straight line or hyperplane**.
- They classify data by computing a **linear combination of input features**.
- Decision boundary is linear:
    - Line (2D)

    - Plane (3D)
    - Hyperplane (higher dimensions)
- Used mainly for **binary classification**.

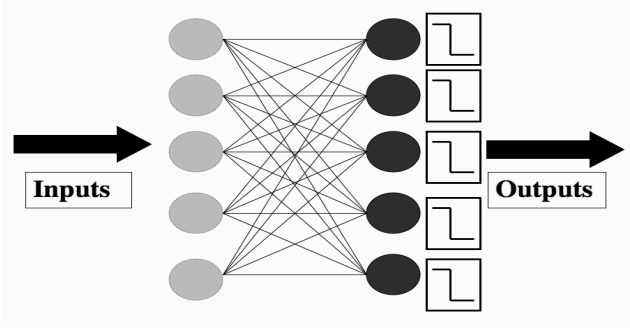
Linear Separability

- Data is said to be **linearly separable** if:
    - A straight line (or hyperplane) can separate classes completely
- In 2D:
    - A line separates two classes
- In higher dimensions:
    - A hyperplane separates classes

Importance

- Perceptron converges **only if data is linearly separable**
- If data is not linearly separable:
    - Perceptron fails to converge
- Example:
    - AND, OR → Linearly separable
    - XOR → Not linearly separable

Perceptron



The Perceptron

$$y = \sum_{i=1}^{n} w_i x_i + b$$

- The Perceptron is a network made up of:
  - **McCulloch and Pitts neurons**
  - A set of **input nodes**
  - **Weighted connections** between inputs and neurons
- Input nodes:
  - Are **not neurons**
  - Only represent how input values are fed into the network
  - Indicate the **dimension of the input vector**
- Neurons:
  - Perform summation and thresholding
  - Include the activation (threshold) internally
- Input nodes are often drawn like neurons, which can be confusing.
- In diagrams, input nodes may be shaded differently for clarity.

Structure of the Perceptron Network

- Each neuron in the Perceptron works **independently**.

- A neuron:
    - Multiplies inputs by its own weights
    - Sums them
    - Compares the result to its threshold
- Neurons do **not depend on outputs of other neurons**.
- Each neuron has its **own set of weights**.
- All neurons share the **same input vector**.
- Number of inputs = **m**
- Number of neurons = **n**
- Inputs and outputs are determined by the **training data**.
- Since supervised learning is used, the target output is known.

Weight Notation

- For a single neuron, weights were written as wi.
- In the Perceptron:
    - Weights are written as wij
    - i → input index
    - j→ neuron index
- Example:
    - w32: weight connecting input 3 to neuron 2
- Weights can be stored in a **2D array**.

Perceptron Output Computation

- Inputs are set according to the input vector.
- Each neuron computes:
    - Weighted sum (Equation 3.1)
    - Activation using threshold function (Equation 3.2)

- Output of the network is a vector of **0s and 1s**.
- Example:
  - Output pattern: (0, 1, 0, 0, 1)
  - Means neuron 2 and neuron 5 fired
- Output is compared with the **target vector**.
- Neurons that produce incorrect outputs must update their weights.

Weight Update Idea

- If a neuron gives the correct output → no change needed.
- If output is incorrect:
  - Weights connected to that neuron must be updated.
- Let:
  - yk = actual output of neuron k
  - tk= target output
- Error can be expressed as:
  - (yk−tk)
- If error is:
  - Negative → neuron should have fired
  - Positive → neuron fired incorrectly
- Weight change depends on both:
  - Error
  - Input value
- Weight update rule:

$$\Delta w_{ik} = (y_k - t_k)x_i$$

Final Weight Update Rule

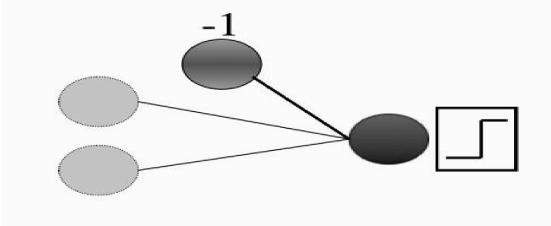
- Learning rate η\etaη controls how much weights are changed.

- Final update equation:
  - $w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j)x_i$
- Training requires **multiple passes** over the dataset.
- Training stops when:
  - All outputs are correct, or
  - Maximum iterations T are reached

Learning Rate (η)

- Learning rate η\etaη controls **speed and stability of learning**.
- If η=1:
  - Large weight changes
  - Network may become unstable
- If η is very small:
  - Learning is slow
  - More stable and noise-resistant
- Typical values:
  - $0.1 < \eta < 0.4$
- Learning rate is a **crucial parameter** in many ML

Bias Input



In biological neurons, firing depends on a **threshold θ**

- Threshold must be adjustable.
- If all inputs are zero:
  - Output depends only on the threshold

- Instead of explicitly updating the threshold:
  - A **bias input** is added
- Bias input:
  - Has a fixed value (usually −1)
  - Has an adjustable weight
- Threshold is fixed at zero.
- Bias weight adjusts firing behavior.
- Bias weight is denoted as:
  - w0j
- Bias allows neurons to behave differently even when inputs are zero.

The Perceptron Learning Algorithm

Phases

- Training Phase
- Recall Phase

Training

- Initialize all weights wij to small random values.
- Repeat for TTT iterations or until all outputs are correct:
  - For each input vector:
    - Compute activation of each neuron:

$$y_j = \begin{cases} 1, & \text{if } \sum w_{ij} x_i > 0 \\ 0, & \text{otherwise} \end{cases}$$

    - Update weights:

$$w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j)x_i$$

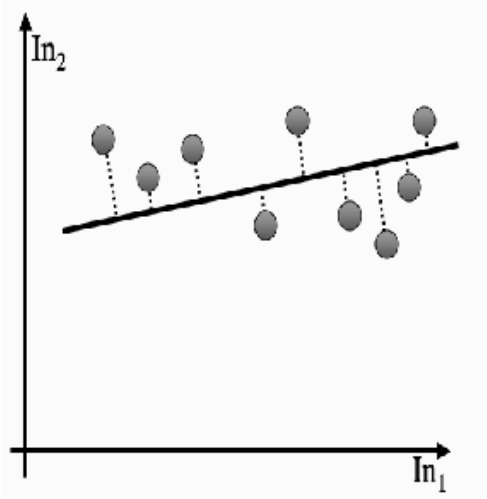
Recall

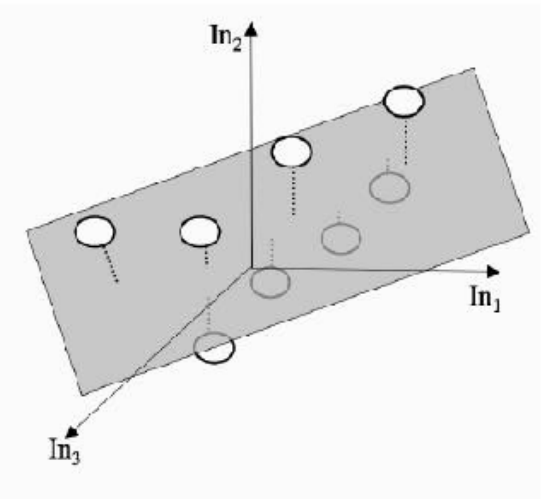
- After training:
    - Compute output using weighted sum and activation
- Recall phase must be **fast**, as it is used frequently.

Linear Regression

Linear Regression

- In statistics and machine learning, problems are broadly divided into:
    - **Regression problems** – fitting a line or surface to data
    - **Classification problems** – separating data into distinct classes
- Regression predicts a **continuous output value**.
- Classification predicts **discrete class labels**.
- Many classification problems can be converted into regression problems.





Regression for Classification

- Classification can be transformed into regression using two methods:
    1. **Indicator Variable Method**
        - Each data point is assigned a variable indicating its class
        - Regression predicts the indicator value
    2. **One-vs-All Regression**
        - Separate regression is performed for each class
        - Output is 1 for examples of the class and 0 for others
- Since classification can be handled using regression, focus is placed on regression methods.

Linear Regression Model

- Linear regression predicts an unknown value yyy from known input values xix_ixi.
- The prediction is a **linear combination of inputs**:
- $$y = \sum_{i=0}^{M} \beta_i x_i$$
- βi are parameters (weights).
- The parameters define:
    - A line (2D)
    - A plane (3D)
    - A hyperplane (higher dimensions)
- Linear regression fits the best line that passes **through or near the data points**.

Least Squares Optimization

- The goal is to find parameters that **best fit the data**.
- This is done by minimizing the **distance between predicted and actual values**.
- Instead of minimizing distance directly:

    - The **sum of squared errors** is minimized
- This method is called **Least Squares Optimization**.
- Error function:

$$\sum_{j=0}^{L}\left(t_j - \sum_{i=0}^{M}\beta_i x_{ij}\right)^2$$

- Objective:
    - Minimize the squared difference between predicted values and actual targets.

Prediction Using Linear Regression

- For a new input vector zzz:
    - Prediction is given by:

      $Y = z\beta$

- This allows prediction of unseen data values.

Linear Regression vs Perceptron

- Both use linear combinations of inputs.
- Key differences:
    - Perceptron:
        - Uses threshold activation
        - Outputs binary values

    - Linear Regression:
        - Produces continuous outputs
        - Uses error minimization

Linear Regression Examples

Regression on Logical OR Function

- Applying linear regression to logical OR:
    - Treats classification as regression
    - Outputs may not be exactly 0 or 1
- Outputs like 1.25 are penalized even if the class is correct.
- Applying a threshold:
    - Values $\geq 0.5 \rightarrow$ class 1
    - Values $< 0.5 \rightarrow$ class 0
- After thresholding:
    - Correct classification is obtained
- Applying regression to XOR:
    - Fails, as XOR is not linearly separable

Training and Evaluation

- Data should be divided into:
    - Training set
    - Testing set
- Training set is used to compute β\betaβ.
- Test set is used to evaluate predictions.
- Confusion matrix is not suitable for regression.
- Performance is measured using:

- **Sum of Squared Errors (SSE)**

- Lower error values indicate better performance.

UNIT-II

Multi-layer Perceptron (MLP)

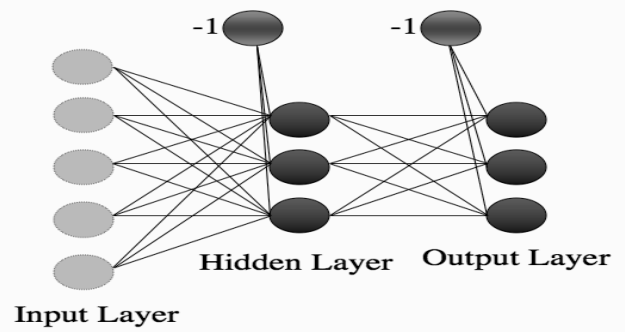
Need for Multi-Layer Networks

- Linear models like the **Perceptron**:
  - Are simple and easy to understand
  - Can represent only **straight lines, planes, or hyperplanes**
- Many real-world problems are **not linearly separable**.
- Feature transformation can sometimes make problems linearly separable:
  - This idea is discussed later in advanced topics
- Another approach is to build **more powerful neural networks**.

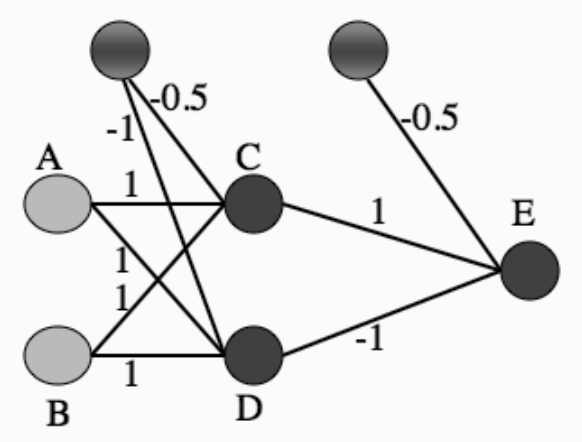
Increasing the Power of Neural Networks

- Learning in neural networks mainly occurs through **weights**.
- To perform more complex computations, we need **more weights**.
- Two possible ways to increase network complexity:
  1. Add **feedback (backward) connections**
     - Leads to recurrent neural networks
     - Less commonly used
  2. Add **more neurons and layers**
     - This leads to **Multi-Layer Perceptrons**
- In MLPs:
  - Extra neurons are added **between input and output layers**
  - These additional layers increase computational power

Structure of a Multi-Layer Perceptron



- An MLP consists of:
  - **Input layer**
  - One or more **hidden layers**
  - **Output layer**
- Neurons are arranged in layers.
- Each neuron in one layer is connected to neurons in the next layer.
- A typical MLP structure is shown in Figure 4.1.



Why Hidden Layers are Important

- Adding hidden layers allows the network to:
  - Represent **non-linear decision boundaries**
  - Solve problems that single-layer perceptrons cannot
- This increased power will be explained in detail later.
- For now, we demonstrate this using the **XOR problem**.

MLP and the XOR Problem

- The XOR problem:
  - Is **not linearly separable**
  - Cannot be solved using a single-layer Perceptron
- A suitably designed MLP **can solve XOR**.
- The MLP for XOR:
  - Has two input nodes
  - One hidden layer with two neurons
  - One output neuron
- The network can be treated as:
  - Two Perceptrons in the hidden layer
  - One Perceptron in the output layer

Working of the XOR MLP (Example)

- Consider input **(1, 0)**:
  - Input A = 1, Input B = 0
- Hidden layer:
  - Neuron C receives weighted sum > threshold → fires
  - Neuron D receives weighted sum ≤ threshold → does not fire
- Output layer:
  - Neuron E receives input from C and D
  - Weighted sum > threshold → fires
- Repeating for all input combinations shows:
  - Output neuron fires **only when inputs are different**
  - This exactly represents the XOR function
- Even though firing/non-firing may be reversed, the logic is correct.

Why Training an MLP is Difficult

- Since MLP can solve XOR:
  - It is clearly more powerful than a Perceptron
- New challenge:
  - **How do we train the network?**
- For output neurons:
  - Error can be computed using known targets
- Problem arises for hidden layers:
  - We do not know the correct output of hidden neurons
  - We cannot directly tell which weights are wrong
- Neurons in intermediate layers are called:
  - **Hidden layers**
- Hidden layers are called “hidden” because:
  - Their correct outputs cannot be observed directly

Importance of the Multi-Layer Perceptron

- The Multi-Layer Perceptron (MLP):
  - Is one of the **most widely used neural networks**
  - Is often treated as a **black box**
- Using MLPs without understanding them:
  - Often leads to poor results
- Understanding how MLPs work requires knowledge of:
  - Statistics
  - Mathematics
  - Computer science
- Learning these foundations helps:
  - Use MLPs effectively
  - Design better learning systems

Going Forwards (Forward Propagation) - Going Backwards: Back-Propagation of Error

Going Forwards (Forward Propagation)

- Training a Multi-Layer Perceptron (MLP) consists of **two main stages**:
  1. **Going forwards** – computing outputs for given inputs and current weights
  2. **Going backwards** – updating weights based on error
- Forward propagation is similar to the **recall phase** of the Perceptron.
- Difference from Perceptron:
  - Computation is done **layer by layer**
  - Outputs of one layer become inputs to the next layer
- Steps in forward propagation:
  - Input values are applied to the input layer
  - Inputs and first-layer weights compute hidden-layer activations
  - Hidden-layer outputs and second-layer weights compute output-layer activations
- This process can be extended to:
  - 2 layers, 3 layers, or many layers
- The forward algorithm does **not change much** even if layers are added.
- After forward propagation:
  - Network outputs are compared with targets
  - Error is computed

Biases

- Every neuron requires a **bias input**.
- Bias is implemented as:
  - An extra input fixed at value **–1**
- Bias weights are trained like all other weights.
- Bias is added:
  - To hidden-layer neurons
  - To output-layer neurons
- Bias allows neurons to:
  - Fire even when all other inputs are zero

Going Backwards: Back-Propagation of Error

- Backward propagation is the **most complex part** of MLP training.
- Errors at the output layer are easy to compute.
- Difficulty arises because:
  - We do not know which weights caused the error
  - Hidden-layer neurons have no direct targets
- Back-propagation:
  - Sends error information **backwards through the network**
  - Is a form of **gradient descent**

Gradient Descent Concept

- Learning is treated as an **error minimization problem**.
- We define an error function and try to **minimize it**.
- Gradient descent:
  - Computes the gradient (slope) of the error function
  - Updates weights in the **negative gradient direction**
- Analogy:
  - Like a ball rolling downhill until it reaches a minimum
- The minimum reached may be:
  - A **local minimum**
  - Not necessarily the global minimum

Error Function for MLP

- Simple error addition can cancel out positive and negative errors.
- To avoid this, **sum-of-squares error** is used:

$$E(t,y) = \frac{1}{2}\sum_{k=1}^{N}(y_k - t_k)^2$$

- Squaring ensures:
  - All errors are positive
  - Large errors are penalized more

Problem with Threshold Activation

- Threshold function is:
  - Discontinuous
  - Not differentiable
- Differentiation is required for gradient descent.
- Solution:
  - Replace threshold function with a **sigmoid function**

Sigmoid Activation Functions

Logistic Sigmoid

$$g(h) = \frac{1}{1 + e^{-\beta h}}$$

- Smooth and differentiable
- Approximates threshold behavior

Hyperbolic Tangent (tanh)

$$g(h) = \tanh(h)$$

- Output range: –1 to +1
- Often useful in practice
- Can be converted to (0,1) range if needed

Backpropagation Idea

- Forward pass:
  - Compute activations of all neurons
- Backward pass:
  - Compute output-layer error terms
  - Propagate errors back to hidden layers
- Uses **chain rule of differentiation**
- Works even with:
  - Multiple hidden layers

Multi-Layer Perceptron Algorithm (Summary)

Network Structure

- L input nodes (+ bias)
- M hidden nodes (+ bias)
- N output nodes
- Weights:
  - (L+1)×M between input and hidden
  - (M+1)×N between hidden and output

Algorithm Steps

1. Initialization

- Initialize all weights to small random values (positive and negative)

2. Training Phase

For each input vector:

Forward Phase

- Compute hidden-layer activations:

$$h_\zeta = \sum x_i v_{i\zeta}$$

$$a_\zeta = g(h_\zeta)$$

- Compute output-layer activations:

$$h_\kappa = \sum a_j w_{j\kappa}$$

$$y_\kappa = g(h_\kappa)$$

Backward Phase

- Compute output-layer error:

$$\delta_o(\kappa) = (y_\kappa - t_\kappa) y_\kappa (1 - y_\kappa)$$

- Compute hidden-layer error:

$$\delta_h(\zeta) = a_\zeta (1 - a_\zeta) \sum w_{\zeta k} \delta_o(k)$$

- Update weights:
  - Output weights
  - Hidden weights

3. Recall Phase

- Use forward phase only

4.2.2 Initializing the Weights

- Weights should be:
  - Small
  - Random
- Large initial weights:
  - Saturate sigmoid
  - Cause slow learning
- Very small weights:
  - Make network almost linear
- Common rule:

$$-\frac{1}{\sqrt{n}} < w < \frac{1}{\sqrt{n}}$$

- Ensures:
  - Uniform learning
  - Balanced weight growth

4.2.3 Different Output Activation Functions

- **Sigmoid output**
  - Used for classification
- **Linear output**
  - Used for regression
  - Activation: g(h)=hg(h) = hg(h)=h
- **Softmax output**
  - Used for multi-class classification
  - Outputs sum to 1
- Changing output activation:
  - Requires changing error derivatives
  - Often paired with **cross-entropy loss**

Multi-layer Perceptron in Practice

The Multi-Layer Perceptron in Practice

- Earlier sections discussed the **design and working** of the MLP.
- This section focuses on **practical choices** required to solve real-world problems.
- MLP is applied to:
  - Regression
  - Classification
  - Time-series prediction
  - Data compression
- Key practical questions:
  - How much training data is required?
  - How many hidden layers and nodes should be used?
  - When should training be stopped?

Amount of Training Data

- For an MLP with **one hidden layer**, total number of weights is:

$$(L+1)M + (M+1)N$$

  where:

  - L = number of input nodes
  - M = number of hidden nodes
  - N = number of output nodes
- Extra **+1** terms represent **bias nodes**.
- These weights are adjustable parameters learned during training.
- Learning is driven by **error signals from training data**.
- More training data:
  - Improves learning
  - Increases training time
- There is **no exact formula** for minimum required data.
- **Rule of thumb**:
  - Number of training examples ≥ **10 × number of weights**
- Because of this:
  - Training MLPs is **computationally expensive**
  - Each example must be presented many times

Number of Hidden Layers

- Two main design choices:
  - Number of hidden nodes
  - Number of hidden layers
- These choices strongly affect performance.
- Important theoretical result:
  - **One hidden layer is sufficient** to approximate any continuous function
  - This is known as the **Universal Approximation Theorem**
- However:
  - One hidden layer may require a **very large number of neurons**
- **Two hidden layers** are:
  - Sufficient for all practical problems
  - Easier to implement efficiently
- There is **no theory** to determine the exact number of hidden nodes.
- Best approach:
  - Experiment with different network sizes
  - Choose the one that gives best validation performance
- MLP can approximate:
  - Any smooth functional mapping

    - Any non-linear decision boundary
- Conceptual explanation:
    - Sigmoid neurons create **ridge-like functions**
    - Combining ridges creates **localized "bump" functions**
    - Linear combinations of bumps can approximate any function
- Figures 4.9 and 4.10 illustrate this idea visually.

When to Stop Learning

- MLP training requires:
    - Multiple passes over the training data
    - Continuous weight updates
- Stopping criteria are important to avoid:
    - Underfitting
    - Overfitting

Common (but imperfect) stopping methods

- Fixed number of iterations
- Stop when training error reaches a minimum
- Stop when error stops decreasing

Problem with these methods

- Network may:
    - Overfit the data
    - Learn noise instead of the underlying function

Validation Set and Early Stopping

- Dataset is divided into:
    - Training set
    - Validation set
- Training set:
    - Used to update weights
- Validation set:
    - Used to evaluate generalisation
- During training:
    - Training error keeps decreasing
    - Validation error decreases initially, then starts increasing
- Increase in validation error indicates **overfitting**
- Training should be stopped at the point where:

    - Validation error is minimum
- This method is called **Early Stopping**
- Early stopping:
    - Improves generalisation
    - Prevents overfitting

.

Examples of using the MLP

Examples of Using the MLP

- The Multi-Layer Perceptron (MLP) is too complex to manually track weight updates.
- Instead of step-by-step calculations, we study **practical demonstrations**.
- MLPs are commonly used for four types of problems:
    1. Regression
    2. Classification
    3. Time-series prediction
    4. Data compression and denoising

Regression Problem

- Goal:
    - Learn an underlying function from sample data
    - Predict values for unseen inputs
- Example function:
    - A simple **sine wave with noise**
- Dataset:
    - One input xxx
    - One output ttt
- Network structure:
    - 1 input node
    - Some hidden nodes (chosen experimentally)
    - 1 output node
- Output layer uses **linear activation**, since output is continuous.
- Steps followed:
    - Generate data
    - Normalise inputs
    - Split data into:
        - Training set
        - Validation set
        - Test set (ratio 50:25:25)

- Training observations:
  - Error decreases over iterations → learning is occurring
- **Early stopping** is used:
  - Training stops when validation error starts increasing
- Choosing number of hidden nodes:
  - Multiple networks trained with different hidden sizes
  - Each size tested multiple times (due to random initialisation)
  - Average validation error is used for comparison
- Result:
  - Small number of hidden nodes (2–10) gives best performance
  - Too many hidden nodes increase error due to overfitting

Classification with the MLP

- Inputs:
  - Feature values (after normalisation)
- Output encoding methods:

Single Output Node

- Uses thresholds on a single output value
- Problems:
  - Artificial class boundaries
  - No confidence information
  - Not scalable for many classes

1-of-N Encoding (Preferred)

- One output neuron per class
- Target vector:
  - All zeros except one position set to 1
  - Example: (0, 0, 1, 0)
- Classification rule:
  - Choose output neuron with **maximum activation**
- Known as:
  - **Hard-max activation**
- Alternative:
  - **Soft-max activation**
  - Outputs sum to 1
  - Gives probability-like outputs

Class Imbalance Issue

- If one class dominates:

    - Network may always predict that class
- Solution:
    - Balance training data
    - Or use **novelty detection**
        - Train only on one class
        - Detect deviations as novel cases

Classification Example: Iris Dataset

- Dataset:
    - Famous **Iris flower dataset**
    - 3 classes
    - 4 input features (sepal & petal measurements)
- Steps:
    - Convert class labels to numbers
    - Normalise input features
    - Convert targets to **1-of-N encoding**
- Dataset size:
    - 150 examples
    - 50 per class (balanced)
- Data split:
    - 50% training
    - 25% validation
    - 25% testing
- Data is **randomised** before splitting
- Training result:
    - Very high classification accuracy
    - Only a few misclassifications
- Demonstrates effectiveness of MLP for multi-class problems

Time-Series Prediction

- Time-series prediction:
    - Predict future values from past observations
- Applications:
    - Weather
    - Stock markets
    - Disease patterns
- Challenges:
    - Long-term trends
    - Seasonal variations
- Training:

    - Inputs taken from earlier time points
    - Target taken from future time point
- Example dataset:
    - Ozone layer thickness data
- Network setup:
    - Treated as a **regression problem**
    - Linear output neurons
- Performance evaluation:
    - Sum-of-squares error
    - No confusion matrix
- Extra tuning required:
    - Number of inputs
    - Values of τ and k

Data Compression: Auto-Associative Network

- Also known as:
    - **Autoencoder**
- Network goal:
    - Reproduce inputs at the output layer
- Key idea:
    - Hidden layer has **fewer neurons than input layer**
- Effect:
    - Hidden layer acts as a **bottleneck**
    - Forces network to compress data
- Result:
    - Lower-dimensional representation
    - Important features retained
    - Noise ignored

Applications

- Image compression
- Data denoising
- Feature extraction

Image Compression Process

- Image converted to 1D vector
- Network trained to reproduce image
- After training:
    - Store hidden-layer activations + second-layer weights
    - This is the compressed representation
- Image reconstruction:

- Feed hidden activations forward
- Output reconstructs original image

Image Denoising

- Network trained on clean images
- Noisy image given as input
- Output is clean version learned by network

Overview – Deriving Back-Propagation

1 What is Back-Propagation?

- Back-Propagation (BP) is a **supervised learning algorithm** used to train **Multi-Layer Perceptrons (MLPs)**.
- It computes how **errors at the output layer** should be distributed back through the network.
- Based on **gradient descent** and the **chain rule of calculus**.
- Goal: **minimise the error (loss) function** by updating weights.

2. Why Back-Propagation is Needed

- In single-layer networks (Perceptron), errors are easy to assign.
- In multi-layer networks:
  - Hidden layer targets are **unknown**
  - We must determine **how much each weight contributed to the final error**
- Back-propagation solves the **credit assignment problem**.

3. Network Assumptions

- Feed-forward neural network
- Differentiable activation functions (e.g., sigmoid, tanh, ReLU*)
- Known target outputs
- Continuous error function

(*Classic derivation assumes sigmoid/tanh)

4. Notation Used

- : input
- : weight from neuron to neuron
- : net input to neuron
- : output of neuron
- : target output
- : learning rate

5. Error (Loss) Function

- Commonly used: **Sum of Squared Errors (SSE)**

- Objective: **minimise E**

6. Key Idea of Derivation

- Adjust weights using **gradient descent**:

- We need to compute **partial derivatives of error with respect to each weight**
- Use **chain rule** to break derivatives into manageable terms

7. Derivation for Output Layer Weights

Using chain rule:

Where:

-
-
-

8. Delta Term (Output Layer)

Define **delta**:

Weight update rule:

9. Derivation for Hidden Layer Weights

- Hidden layer does not have direct error
- Error is **propagated backward** from output layer

Hidden layer delta:

10. Weight Update Rule (Hidden Layer)

11. Complete Back-Propagation Algorithm

1. Initialise weights randomly
2. Forward pass:
    - Compute outputs layer by layer
3. Compute error at output layer
4. Backward pass:
    - Compute delta for output layer
    - Compute delta for hidden layers
5. Update all weights
6. Repeat until:
    - Error converges, or
    - Maximum epochs reached

12. Key Properties

- Uses **gradient descent**
- Works only with **differentiable activation functions**
- Sensitive to:

    - Learning rate
    - Initial weights
    - Local minima
- Computationally efficient for deep networks

13. Advantages

- Enables training of deep neural networks
- General-purpose learning algorithm
- Handles non-linear decision boundaries

14. Limitations

- Can converge slowly
- May get stuck in local minima
- Requires careful tuning of hyperparameters

Radial Basis Functions and Splines

Receptive Fields

- Nodes are viewed as points in **weight space**
- Activation depends on **distance between input vector and node**
- Closer input → stronger activation
- Farther input → weaker or zero activation

Radial Functions

- Since activation depends only on distance:
    - Functions are **radially symmetric**
- Depend on:

$$\|x - w\|_2$$

- Most common choice:

- **Gaussian function**

Gaussian Activation Function

$$g(x, w, \sigma) = \exp\left(-\frac{\|x - w\|^2}{2\sigma^2}\right)$$

- w: centre of receptive field
- σ: width (spread)
- Large σ
  - Broad receptive field
- Small σ:
  - Narrow receptive field (high locality)

Key Properties of RBFs

- Local activation
- Smooth and differentiable
- Strong response near centre
- Weak response far away
- Require **input normalisation**

Radial Basis Function (RBF) Network

Architecture

- Three layers:
  1. Input layer
  2. Hidden layer (RBF neurons)
  3. Output layer (linear or perceptron)
- Hidden layer:
  - Uses Gaussian activations
- Output layer:
  - Combines activations linearly

Why RBF is Better than Perceptron

- Input to output layer is **non-linear transformation**
- Network becomes a **universal approximator**
- Uses **local decision regions**
- Unlike MLP:
    - RBF neurons respond locally
    - MLP neurons separate space using **global hyperplanes**

Universal Approximation Property

- If RBF centres cover input space sufficiently:
    - Any continuous function can be approximated
- More RBFs → finer approximation
- Larger σ\sigmaσ → smoother approximation

MLP vs RBF

Feature	MLP	RBF
Hidden nodes	Global	Local
Activation	Inner product	Distance-based
Layers	Multiple non-linear layers	One non-linear layer
Learning	Gradient descent	Hybrid (unsupervised + supervised)

Step 1: Position RBF Centres

Two common methods:

1. **Random selection**
    - Choose random training samples as centres
2. **k-means clustering**
    - Centres placed at cluster means
    - More structured and effective

Step 2: Compute RBF Activations

- Use Gaussian function (Equation 5.1)
- Produces hidden layer output matrix GGG

Step 3: Train Output Weights

Two options:

(a) Perceptron Learning

- Treat RBF activations as inputs
- Use standard perceptron rule

(b) Pseudo-Inverse Method (Preferred)

- Faster, one-step training
- Uses:

$$W = G^{+}t$$

- Where:
  - G+ is pseudo-inverse
  - Computed using np.linalg.pinv()

Choosing Width (σ)

- Common heuristic:

$$\sigma = \frac{d}{\sqrt{2M}}$$

- d: maximum distance between centres
- M: number of RBF nodes

Normalised RBFs

- Ensures at least one neuron fires
- Similar to soft-max

$$g_i(x) = \frac{\exp(-\|x - w_i\|^2 / 2\sigma^2)}{\sum_j \exp(-\|x - w_j\|^2 / 2\sigma^2)}$$

Splines

A spline is a piecewise polynomial function used to smoothly approximate or interpolate data.

Cubic Splines

- Most commonly used spline
- Basis functions:
  - $1, x, x^2, x^3$
  - Plus knot-based cubic terms
- Ensures:
  - Continuity of function
  - Continuity of first and second derivatives

Fitting Splines to Data

- Uses **least squares minimisation**

$$E = \sum (y_i - f(x_i))^2$$

- Linear in parameters
- Solved using:
  - np.linalg.lstsq()

Smoothing Splines

- Exact interpolation → overfitting
- Solution:
  - Add smoothness penalty

$$E = \sum (y_i - f(x_i))^2 + \lambda \int (f''(x))^2 dx$$

- λ:
  - Controls trade-off
  - Large → smoother curve
  - Small → closer fit

Curse of Dimensionality

Dimensionality Reduction

Why Dimensionality Reduction is Needed

- Humans can easily visualize:
  - 2D and 3D data only
- High-dimensional data causes:
  - **Curse of Dimensionality**
  - Need for **more training data**
  - Increased **computational cost**
- Benefits:
  - Removes noise
  - Improves learning performance
  - Simplifies datasets
  - Makes results easier to interpret
- Enables visualization (e.g., SOM, PCA plots)

Ways to Perform Dimensionality Reduction

1. Feature Selection

- Select useful features
- Remove irrelevant or redundant features
- Based on:
  - Correlation with output
  - Domain knowledge
- Types:
  - **Constructive**: start with none, add features
  - **Destructive**: remove features and test impact

- Search problem:
  - For d features → 2d−12^d - 12d−1 combinations
- Uses greedy methods due to high complexity

2. Feature Derivation (Feature Extraction)

- Create new features from old ones
- Uses:
  - Linear transformations
  - Rotations
  - Projections
- Combines features to identify useful dimensions
- Examples:
  - PCA
  - LDA

3. Clustering-based Reduction

- Group similar datapoints
- Use cluster structure to reduce dimensions
- Mostly unsupervised

Interpolations

Interpolations

Interpolation is the process of **estimating unknown values** between known data points.

Given a set of data points, interpolation finds a function that **passes exactly through all points**.

Given data:

$$(x_1, t_1), (x_2, t_2), \ldots, (x_n, t_n)$$

Find a function $f(x)$ such that:

$$f(x_i) = t_i \quad \forall i$$

Types of Interpolation

a) Linear Interpolation

- Connects data points using straight lines
- Simple and fast
- Not smooth at boundaries

b) Polynomial Interpolation

- Uses a single polynomial passing through all points
- High-degree polynomials can cause overfitting
- Poor behaviour at boundaries (Runge's phenomenon)

c) Piecewise Interpolation

- Data range divided into segments
- Each segment approximated separately
- More stable than global polynomials

Basis Functions

Basis functions are building blocks used to represent complex functions.

Any function is expressed as a weighted sum of basis functions.

$$f(x) = \sum_{i=1}^{M} w_i \phi_i(x)$$

Where:

- $\phi_i(x)$ → basis functions
- w_i → weights/parameters

Purpose of Basis Functions

- Transform input space
- Simplify learning by making the model linear in parameters
- Allow modelling of non-linear relationships

4. Types of Basis Functions

a) Polynomial Basis Functions

- **Simple and easy to implement**
- **Risk of overfitting for high degree**

b) Radial Basis Functions (RBF)

- **Localised response**
- **Depends on distance from centre**
- **Commonly Gaussian**

c) Spline Basis Functions

- **Piecewise polynomials**
- **Cubic splines are most popular**

- **Ensure smoothness at knot points**

5. Choice of Basis Functions

- **Determines model complexity**
- **Too few → underfitting**
- **Too many → overfitting**

6. Regularisation with Basis Functions

- **Prevents overfitting**
- **Controls smoothness**

7. Relation to Machine Learning

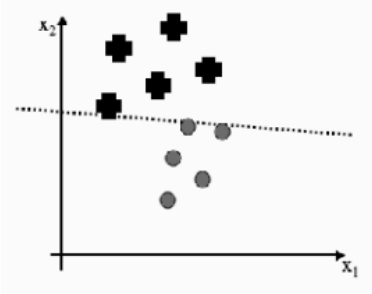
- **Used in:**
    - **Linear regression**
    - **RBF networks**
    - **Support Vector Machines (kernels)**
    - **Splines**

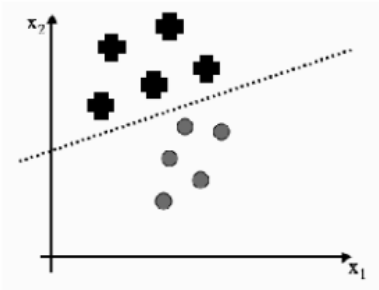
8. Advantages

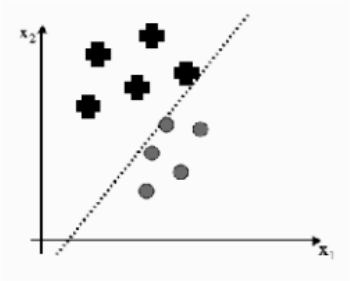
- **Flexible representation**
- **Linear optimisation**
- **Strong theoretical foundation**

Support Vector Machines

Support Vector Machines (SVM)

- 



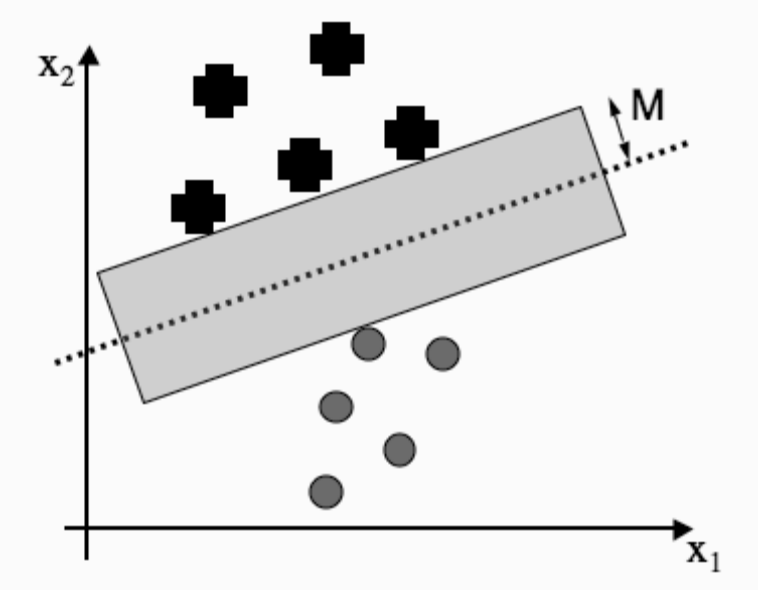


  Perceptron:
  - Can only learn **linear classifiers**
  - Fails for **non-linearly separable data** (e.g., XOR)
- SVM:
  - Introduced by **Vapnik (1992)**
  - Uses **kernel methods** to handle non-linear data
  - One of the most powerful classification algorithms
- Key ideas:
  - Optimal separation
  - Maximum margin
  - Support vectors
  - Kernel trick

Optimal Separation

- Many lines can separate data
- SVM chooses the **best possible line**
- Best line:
  - Lies **midway between classes**
  - Maximizes separation
- Goal:
  - Improve **generalization on unseen data**

Margin and Support Vectors



Margin

- Margin = distance between:
  - Decision boundary
  - Closest data points
- Larger margin ⇒ better classifier
- Margin width:

$$M = \frac{1}{||w||}$$

Support Vectors

- Data points **closest to the decision boundary**
- Only these points affect:
  - Position of the classifier
- After training:
  - Non-support vectors can be discarded
- SVM gives a **sparse representation**

Decision Boundary

$$w^T x + b = 0$$

- Class +1 if:

$$w^T x + b \geq 1$$

- Class −1 if:

$$w^T x + b \leq -1$$

Soft Margin SVM (Slack Variables)

Problem

- Real-world data is **not perfectly separable**

Solution

- Introduce slack variables ξ_i

$$t_i(w^T x_i + b) \geq 1 - \xi_i$$

New Objective Function

$$\min\left(\frac{1}{2}||w||^2 + C\sum \xi_i\right)$$

- **C**:
  - Controls trade-off
  - Small C → larger margin, more errors
  - Large C → fewer errors, smaller margin

Kernels

Motivation

- Transform data into higher dimensions
- Make non-linear problems **linearly separable**

Feature Mapping

$$x \rightarrow \phi(x)$$

- Explicit computation is expensive

Kernel Trick

- Avoid computing φ(x) explicitly
- Use:

$$K(x, y) = \phi(x)^T \phi(y)$$

- All computations done in **original space**

Kernelized Decision Function

f(z)=∑iλitiK(xi,z)+bf(z) = \sum_i \lambda_i t_i K(x_i, z) + bf(z)=i∑ λitiK(xi,z)+b

Common Kernel Functions

1. Linear Kernel

$$K(x, y) = x^T y$$

---

2. Polynomial Kernel

$$K(x, y) = (1 + x^T y)^s$$

---

3. Sigmoid Kernel

$$K(x, y) = \tanh(\kappa x^T y - \delta)$$

---

4. RBF (Gaussian) Kernel

$$K(x, y) = \exp\left(\frac{-||x - y||^2}{2\sigma^2}\right)$$

SVM Algorithm

Steps

1. Initialization

- Choose kernel
- Compute kernel matrix K

2. Training

- Solve quadratic programming problem
- Identify support vectors
- Compute bias b

3. Classification

- For test point z:

$$f(z) = \sum \lambda_i t_i K(x_i, z) + b$$

Computational Cost

- Kernel computation: O(m2n)O(m^2 n)O(m2n)
- Solver: O(m3)O(m^3)O(m3)
- Not suitable for **very large datasets**

SVM for Regression (SVR)

- Uses **ε-insensitive loss**
- Errors within ε are ignored

Prediction

$$f(z) = \sum (\mu_i - \lambda_i) K(x_i, z) + b$$

Advantages of SVR

- Sparse model
- Robust to noise
- Good generalization

UNIT-III

Learning with Trees (Decision Trees)

- Decision Trees are one of the most **popular and interpretable** machine learning models.
- Based on **binary tree data structure**.
- Advantages:
  - Fast prediction time: **O(log N)**
  - Easy to understand and explain (not a black box)
  - Can be converted into **if–then rules**
  - Handles **categorical data, missing data, and noise**
- Widely used in:
  - Expert systems
  - Customer support systems
  - Medical diagnosis
  - Credit risk analysis

Decision Trees

- A decision tree:
  - Starts at a **root node**
  - Internal nodes → tests on features
  - Branches → feature values
  - Leaf nodes → class labels / outputs
- Classification is done by:
  - Traversing the tree from root to leaf
- Can be easily converted into **logical rules**:
  - IF condition THEN decision

Key Characteristics

- Transparent decision-making
- Greedy learning strategy
- Uses **feature-based splitting**
- Suitable for both **classification and regression**



One of the reasons that decision trees are popular is that we can turn them into a set of logical disjunctions (if ... then rules) that then go into program code very simply—the first part of the tree above can be turned into:

• if *there is a party* then *go to it*

• if *there is not a party* and *you have an urgent deadline* then *study*

• etc.

Constructing Decision Trees

Basic Idea

- Build tree **top-down**
- At each step:
  - Select the **most informative feature**
  - Split data based on that feature
- Uses a **greedy heuristic** (local optimum at each step)

Popular Decision Tree Algorithms

- **ID3** – Uses Information Gain
- **C4.5** – Extension of ID3 with pruning

- **CART** – Classification and Regression Trees

ID3 Algorithm

Steps

1. If all examples belong to same class:
    - Create a leaf node
2. If no features remain:
    - Assign most common class
3. Else:
    - Compute Information Gain for all features
    - Select feature with highest gain
    - Split dataset and recurse

Stopping Conditions

- All examples have same label
- No features left
- Empty dataset

Inductive Bias

- ID3 prefers:
    - Smaller trees
    - Simpler explanations
- Based on:
    - **Occam's Razor**
    - **Minimum Description Length (MDL)** principle

Handling Noise and Missing Data

- Noise:
    - Tree assigns majority class
- Missing data:
    - Skip feature
    - Average over possible branches
- Advantage over neural networks

Overfitting in Decision Trees

Problem

- Tree may grow too deep
- Memorizes training data

Solutions

1. Early Stopping

- Limit depth
- Stop splitting when gain is small

2. Pruning

- Remove unnecessary branches
- Improves generalization

Pruning Methods

- **Reduced Error Pruning**
- **Rule Post-Pruning (C4.5)**:
    - Convert tree to rules
    - Remove conditions if accuracy improves

CART (Classification and Regression Trees)

- Always builds **binary trees**
- Works for:
    - Classification
    - Regression
- Uses different impurity measures

Gini Impurity

- Measures probability of misclassification

$$Gini = 1 - \sum p_i^2$$

Properties:

- 0 → pure node
- Faster than entropy
- Commonly used in CART

Weighted Gini Impurity

- Includes misclassification cost

$$G = \sum \lambda_{ij} N(i) N(j)$$

- Useful when:
  - Some errors are more costly

Regression Trees

- Used when output is **continuous**
- At each leaf:
  - Output = mean of target values
- Splits chosen by minimizing:
  - **Sum of Squared Errors (SSE)**

Classification Example (ID3)

- Compute entropy of dataset
- Calculate information gain for each feature
- Select feature with highest gain as root
- Repeat recursively
- Produces final decision tree

Advantages of Decision Trees

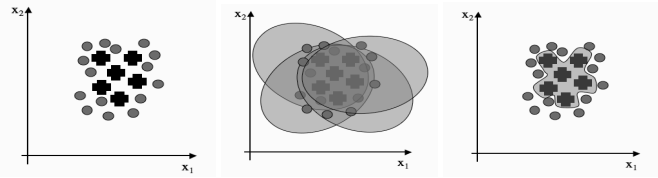
- Easy to interpret
- Handles mixed data types
- Requires little data preprocessing
- Fast inference
- Handles missing values

Disadvantages of Decision Trees

- Prone to overfitting
- Greedy algorithm (local optimum)
- Unstable to small data changes

Ensemble Learning

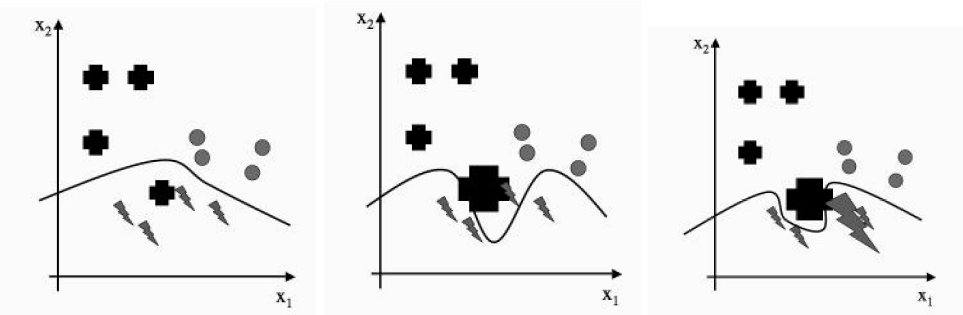
By combining lots of simple classifiers (here that simply put an elliptical decision boundary onto the data), the decision boundary can be made much more com-plicated, enabling the difficult separation of the pluses from the circles.



- Ensemble Learning is based on the idea that **multiple learners combined together perform better than a single learner**.
- Inspired by the saying: *"Two heads are better than one."*
- Instead of relying on one classifier, we use a **committee of classifiers**.
- Each classifier:
  - Learns slightly different aspects of the data
  - Makes its own prediction
- Final decision is obtained by **combining their outputs**.
- Common base learners:
  - Decision Trees
  - Weak classifiers
- Ensemble learning improves:
  - Accuracy
  - Robustness
  - Generalisation

Boosting

As points are misclassified, so their weights increase in boosting (shown by the datapoint getting larger), which makes the importance of those datapoints increase, making the classifiers pay more attention to them



- Combine many **weak learners** into a **strong learner**
- Weak learner:
    - Performs only slightly better than random guessing
- Boosting adaptively focuses on:
    - Hard-to-classify examples

Key Characteristics

- Sequential learning
- Each classifier depends on previous ones
- Misclassified points receive **higher importance**
- Final classifier is a **weighted combination**

AdaBoost (Adaptive Boosting)

- Each data point has a **weight**
- Initially:
    - All weights are equal (1/N)
- After each iteration:
    - Misclassified points get **higher weights**
    - Correctly classified points get lower importance
- Forces next classifier to focus on difficult cases

AdaBoost Algorithm (Steps)

1. Initialise weights:

$$w_i = \frac{1}{N}$$

2. For each iteration:
    - Train weak classifier using weighted data
    - Compute weighted error:

$$E = \sum w_i \text{ (misclassified points)}$$

    - Compute classifier weight:

$$\alpha = \ln\left(\frac{1-E}{E}\right)$$

    - Increase weights of misclassified points
    - Normalise weights
3. Final classifier:
    - Weighted vote of all classifiers

Stopping Conditions

- Error becomes 0
- Error ≥ 0.5
- Maximum iterations reached
- One datapoint dominates the weights

Advantages of AdaBoost

- Very high accuracy
- Automatically focuses on hard examples
- Works well with simple classifiers
- Resistant to overfitting (in many cases)

Limitations

- Sensitive to noisy data
- Sequential → not easily parallelised
- Weak learners must support weighted data

Bagging (Bootstrap Aggregating)

- Create multiple datasets using **bootstrap sampling**
- Train one classifier per dataset
- Combine outputs using **majority vote**

Bootstrap Sampling

- Sampling **with replacement**
- Each sample:
  - Same size as original dataset
  - Contains duplicates
- Ensures classifiers are **different**

Bagging Algorithm

1. Generate B bootstrap samples
2. Train a classifier on each sample
3. Combine predictions by:
   - Majority voting (classification)
   - Averaging (regression)

Advantages of Bagging

- Reduces variance
- Improves stability
- Simple to implement
- Parallelisable

Limitations

- Does not reduce bias
- Needs many classifiers
- Less effective for very strong learners

Subagging

- Subsample Aggregating
- Sampling **without replacement**
- Smaller datasets than original
- Faster than bagging
- Comparable performance in practice

Random Forests (Bagging)

- Ensemble of **decision trees**
- Combines:
    - Bagging
    - Random feature selection

How Random Forests Create Randomness

1. **Data randomness**
    - Bootstrap samples for each tree
2. **Feature randomness**
    - Random subset of features at each split

Random Forest Training Algorithm

1. For each tree:
    - Take bootstrap sample
    - Grow a decision tree
    - At each node:
        - Randomly select m features
        - Choose best split among them
2. No pruning required
3. Combine trees by:
    - Majority vote (classification)
    - Mean output (regression)

Key Properties

- High accuracy
- Robust to noise
- Low overfitting

- Handles large datasets
- Embarrassingly parallel
- Built-in error estimation using:
    - **Out-of-bag (OOB) samples**

Parameters

- Number of trees
- Number of features (m) per split
    - Typically d\sqrt{d}d
- Maximum tree depth

Random Forest (Bagging) vs Boosting

Feature	Boosting	Random Forest
Learning	Sequential	Parallel
Focus	Hard examples	Randomness
Speed	Slower	Faster
Noise sensitivity	High	Low
Scalability	Limited	Excellent

Different Ways to Combine Classifiers

Voting Methods

- Majority voting
- Weighted voting
- Consensus voting (only if all agree)

Probability Insight

- If each classifier has accuracy > 50%
- Ensemble accuracy → 1 as classifiers increase
- Based on binomial probability

Combining for Regression

- Mean of outputs
- Median (robust to outliers)
- Median-based bagging = **Bragging**

Mixture of Experts

- Learns **how to combine classifiers**
- Uses:
    - Experts (classifiers)
    - Gating networks
- Gates assign weights dynamically based on input
- Soft decision boundaries
- Trained using:
    - EM algorithm
    - Gradient descent

Interpretation

- Soft version of decision trees
- Similar to RBF networks
- Different experts handle different regions of input space

Advantages of Ensemble Learning

- Higher accuracy
- Better generalisation
- Robust to noise
- Reduces variance and overfitting

Disadvantages

- Increased computation
- Less interpretability
- More memory usage